UNIVERSIDAD
Panamericana

PROGRAMACIÓN AVANZADA

Sesión 13

Torres de Hanoi: Solución Recursiva

→ NOTAS DE CLASE

Universidad Panamericana
Dr. Miguel Alcaraz Rivera • malcaraz@up.edu.mx

2026

Índice

1 El concurso de diseño **2**

2 ¿Se puede resolver Hanoi en seis líneas? **2**

3 El árbol de llamadas **3**

4 El call stack durante la ejecución **3**

5 Verificación: contando movimientos **4**

6 La actividad de hoy **4**
6.1 Lo que debes implementar 4
6.2 Criterios de evaluación 5

7 Ejercicios **5**

8 Resumen **6**

El concurso de diseño

Antes de ver el algoritmo de hoy, el grupo elige el mejor diseño de la sesión 12. Las reglas del concurso son simples:

- Cada persona vota por **exactamente tres** equipos.
- Los criterios son: diseño visual atractivo **y** código legible. Los dos cuentan. Un diseño bonito con código ilegible no gana.
- El equipo con más votos comparte su código en pantalla.
- Ese código se usará como punto de partida para la actividad de hoy.

> Cuando estés votando, pregúntate: ¿entiendo lo que hace cada función con solo leer el nombre? ¿Las torres se ven bien en todos los estados? Un voto bien razonado pesa más que uno al azar.

¿Se puede resolver Hanoi en seis líneas?

La sesión 12 te tomó 50 minutos de trabajo en equipo para mover los discos a mano. Hoy verás que **la solución recursiva tiene seis líneas de código**.

¿Cómo es posible? Porque la descripción recursiva del problema *ya es* el algoritmo. Recuerda la lógica de la sesión 12:

> Para mover n discos de A a C usando B como auxiliar:
> 1. Mueve los $n-1$ discos superiores de A a B (usando C como auxiliar).
> 2. Mueve el disco n (el más grande) de A a C.
> 3. Mueve los $n-1$ discos de B a C (usando A como auxiliar).

Eso en C++ se escribe exactamente así:

```
void hanoi(int n, char origen, char destino, char aux,
           vector<int>& torA, vector<int>& torB, vector<int>&
               torC) {
    if (n == 0) return;
    hanoi(n-1, origen, aux, destino, torA, torB, torC);
    moverDisco(origen, destino, torA, torB, torC);
    hanoi(n-1, aux, destino, origen, torA, torB, torC);
}
```

El caso base es $n = 0$: no hay nada que mover, se regresa de inmediato. Todo lo demás es consecuencia de los tres pasos descritos arriba.

> ✔ **Recuerda**
>
> La recursión no es magia: es la misma lógica que aplicaste a mano en la sesión 12, traducida directamente a código. La computadora simplemente la ejecuta $2^n - 1$ veces sin cansarse.

El árbol de llamadas

Cada llamada a `hanoi(n, ...)` genera **dos llamadas** a `hanoi(n-1, ...)`. El árbol de recursión para $n = 3$ tiene esta forma:

```
                    hanoi(3, A, C, B)
                   /                 \
        hanoi(2, A, B, C)        hanoi(2, B, C, A)
         /          \              /          \
hanoi(1,A,C,B) hanoi(1,C,B,A)  hanoi(1,B,A,C) hanoi(1,A,C,B)
  /    \          /    \          /    \          /    \
h(0)  h(0)    h(0)  h(0)      h(0)  h(0)      h(0)  h(0)
```

Las hojas (`h(0)`) son los casos base. Los nodos internos son las llamadas que hacen trabajo real (`moverDisco`).

Contando los nodos internos del árbol:

n	Hojas	Nodos internos	Movimientos
1	2	1	1
2	4	3	3
3	8	7	7
4	16	15	15
n	2^n	$2^n - 1$	$2^n - 1$

Cada nodo interno corresponde exactamente a un movimiento. Por eso $M(n) = 2^n - 1$: es el número de nodos internos de un árbol binario completo de profundidad n.

El call stack durante la ejecución

La profundidad del call stack en cualquier momento es **como máximo** n, porque cada llamada espera a que su primera sub-llamada ($n - 1$) termine antes de continuar. Nunca hay más de n frames activos simultáneamente:

```
Momento en que se mueve el disco 3 (paso 4 de 7):

  hanoi(3, A→C, B)   ← frame más antiguo, esperando
    hanoi(2, A→B, C) ← ya terminó
    [moverDisco A→C] ← ← línea activa aquí
    hanoi(2, B→C, A) ← aún no llamada
```

La profundidad máxima es n, no 2^n. Para $n = 64$ el stack tendría solo 64 frames; lo que explota es el *tiempo*, no la memoria.

La función `hanoi` tiene:

- **Complejidad temporal:** $O(2^n)$ — el número de llamadas totales crece exponencialmente con n.

- **Complejidad espacial:** $O(n)$ — la profundidad máxima del call stack en cualquier momento es n.

Muchos algoritmos recursivos tienen complejidad espacial proporcional a la *profundidad del árbol*, no al número total de llamadas.

Verificación: contando movimientos

Una forma de verificar que el algoritmo es correcto es contar cuántas veces se llama `moverDisco`. Si el programa mueve n discos, el contador debe ser exactamente $2^n - 1$.

```
int movimientos = 0;

void hanoi(int n, char origen, char destino, char aux,
           vector<int>& torA, vector<int>& torB, vector<int>&
              torC) {
    if (n == 0) return;
    hanoi(n-1, origen, aux, destino, torA, torB, torC);
    moverDisco(origen, destino, torA, torB, torC);
    movimientos++;
    hanoi(n-1, aux, destino, origen, torA, torB, torC);
}
```

Al terminar, `movimientos == pow(2, n) - 1`.

La actividad de hoy

Trabajas con el código ganador del concurso como base.

• Lo que debes implementar

1. **La función `hanoi`** que resuelve el problema automáticamente, usando `moverDisco` del código base para actualizar el estado de las torres.
2. **Imprimir las torres después de cada movimiento** para que se pueda seguir la solución visualmente.
3. **Contar y mostrar** el número de movimientos al final y verificar que sea $2^n - 1$.

- **Criterios de evaluación**

Criterio	Descripción	Peso
Corrección	El programa mueve todos los discos de A a C respetando las reglas en cada paso	40 %
Conteo	Al terminar, imprime el número de movimientos y verifica que sea $2^n - 1$	20 %
Visualización	Las torres se muestran (o pueden mostrarse) después de cada movimiento	20 %
Integración	La función recursiva usa `moverDisco` del código base sin reescribirla	20 %
	Total	100 %

Muchas soluciones de Hanoi en internet imprimen directamente los movimientos (`cout « "Mueve de " « A « " a " « C`). Eso no es lo que se pide: el objetivo es que la función use `moverDisco` para actualizar el `vector`, de modo que las torres se puedan dibujar con el diseño elegante de la sesión 12.

Ejercicios

1. Traza a mano el árbol de recursión de `hanoi(2, A, C, B)`. Indica en qué orden se hacen los movimientos y qué disco se mueve en cada uno.

2. ¿Cuántas veces se llama `hanoi` en total (incluyendo las llamadas con $n = 0$) cuando se invoca `hanoi(4, ...)`? *(No olvides las hojas del árbol.)*

3. Modifica `hanoi` para que reciba un parámetro `int nivel` (inicialmente 0) y muestre una indentación proporcional al nivel antes de cada movimiento, igual que hiciste con `sumaTraza` en la sesión 11.

4. ¿Es posible implementar Hanoi de forma iterativa (sin recursión)? Si crees que sí, describe en palabras cómo sería la lógica. Si crees que no, justifica por qué.

5. **Desafío:** Modifica el programa para que el usuario pueda elegir el número de discos n al inicio. Añade un límite: si $n > 20$, muestra solo el conteo de movimientos sin dibujar las torres (para no inundar la terminal).

Reto extra — Fibonacci y la razón áurea

¿Terminaste antes que todos? Este reto conecta la recursión con uno de los números más famosos de las matemáticas.

La **sucesión de Fibonacci** se define recursivamente:

$$F(0) = 0, \quad F(1) = 1, \quad F(n) = F(n-1) + F(n-2) \quad (n \geq 2).$$

Al dividir términos consecutivos ocurre algo sorprendente:

$$\frac{F(n)}{F(n-1)} \xrightarrow{n\to\infty} \varphi = \frac{1+\sqrt{5}}{2} \approx 1.6180339\ldots$$

Este número, φ, se llama la **razón áurea** (*golden ratio*). Aparece en arquitectura, arte, biología y, resulta que también, en la recursión.

Lo que debes hacer:

1. Implementa `fib(n)` de forma recursiva.
2. Imprime la siguiente tabla para n de 2 a 20, usando `double` para la razón:

```
 n      F(n)      F(n)/F(n-1)
 2         1      1.000000
 3         2      2.000000
 4         3      1.500000
 5         5      1.666667
...
20      6765      1.618034
```

   ¿A qué valor converge la razón?

3. Dibuja el árbol de llamadas de `fib(5)` y compáralo con el de `hanoi(3)`. ¿Qué diferencia importante notas?
   Pista: ¿algún subproblema aparece más de una vez?

La versión recursiva ingenua de `fib` repite subproblemas: `fib(3)` se calcula múltiples veces dentro de `fib(5)`. Esto la hace $O(\varphi^n)$ en tiempo, *más lenta* que Hanoi para el mismo n, aunque Hanoi tiene $O(2^n)$ movimientos reales. La diferencia es que Hanoi no repite subproblemas: cada llamada hace trabajo único. Esa ineficiencia de Fibonacci tiene solución —se llama **memoización**— y la verás en sesiones posteriores.

Resumen

✔ Recuerda

- La solución recursiva de Hanoi sigue exactamente los mismos tres pasos que aplicaste a mano en la sesión 12.
- El código tiene **una** condición base ($n = 0$) y **tres** líneas de trabajo: sub-llamada, mover, sub-llamada.
- El árbol de recursión tiene $2^n - 1$ nodos internos, uno por movimiento.
- La profundidad del call stack es $O(n)$; el tiempo es $O(2^n)$.
- El número de movimientos se puede verificar: siempre debe ser exactamente $2^n - 1$.